Project Sentinel

Privacy Automation Platform

Technical Specification

System Architecture, Data Model, and Implementation Plan

Document Version	0.2
Status	Concept Specification
Author	Jan van Hout
Date	August 24, 2026

Table of Contents

Purpose of This Document

This document records the concrete technical stack, architecture, and implementation decisions made for Project Sentinel, organized to mirror the phase structure defined in Section 32 of the Project Sentinel Specification (v0.2). Each phase is documented once the relevant decisions have been finalized. This is a living document: new sections are appended as each development phase is worked through, and earlier sections may be revised if later phases reveal a need to reconsider prior decisions. Every decision below is made in service of the priority order and philosophy established in the base specification (Sections 2.1–2.3): legal compliance, privacy, extensibility, maximum automation, transparency, community maintenance, and speed, in that order.

CHAPTER 1

Requirements Specification and Governance Foundation

This chapter documents the two deliverables of Phase 1 that are not the Requirements Specification itself: the Contribution Guidelines and the Initial Governance Model, along with a clarification of this Technical Specification's own status relative to the freeze Phase 1 applies to the base specification.

1.1 Requirements Specification Status

Decision: The Project Requirements Specification (Spec v0.2) is treated as frozen for the purposes of Phase 1, per Section 32's instruction that the specification be finalized before architectural work begins. This Technical Specification, by contrast, remains a living document throughout development, as already stated in its own Purpose section (Section 0), and is not subject to the same freeze.
Rationale: Section 32 requires the Requirements Specification to be reviewed, refined, and frozen before architectural work begins, so that every subsequent phase is building against a stable, unambiguous definition of the project. That requirement is specific to the Requirements Specification itself, the document defining what the project is and why. This Technical Specification instead documents how those requirements are implemented, and its own stated purpose (Section 0) is to be updated as each phase is worked through and revised where later phases reveal a need to reconsider earlier decisions. Applying the same freeze to this document would contradict its own declared purpose, and would work against the practical value of treating it as a living record. For instance, the consistency review that corrected citation ambiguity and content gaps across Phases 3, 5, 6, and 7 was only possible because this document is not frozen.

1.2 Contribution Guidelines

Decision: A CONTRIBUTING.md file is maintained in the repository, covering the code style and tooling expectations already established (Sections 4.4 to 4.8: Ruff, mypy, pre-commit hooks), the pull request process and checklist (Section 4.10), and separate guidance for proposing company-database corrections (Section 14) and plugin submissions (Sections 12.1 to 12.4), since these follow different review paths than application code changes.
Rationale: Consolidating contribution guidance into one document gives a new contributor a single starting point, while still directing them to the correct process for the specific kind of contribution they're making. A code change, a company-database correction, and a plugin submission are reviewed differently (Sections 14,

12.1), and conflating them into one generic guide would leave a contributor unsure which rules actually apply to their submission.

1.3 Initial Governance Model

Decision: For v1.0, a single maintainer holds final decision authority over the project, including architectural decisions, release approval, and moderator/plugin-registry trust decisions (Sections 3.9, 12.1). A path toward a broader maintainer team is acknowledged as a future direction but is not scoped or formalized as part of v1.0.
Rationale: A single-maintainer model matches the project's actual current scale and avoids introducing formal governance overhead, such as voting procedures or maintainer onboarding criteria, before there is a team to apply it to. This is consistent with the project's own community-governance design elsewhere: Section 14's reputation-weighted voting and moderator threshold, and Section 12.1's trusted-author key registry, both describe processes for community contributions, not decisions about the project's core direction. Those remain the maintainer's responsibility until a broader governance model is deliberately established. Leaving that expansion path acknowledged but unscoped keeps this section honest about the project's current state rather than describing a governance structure that doesn't yet exist.

CHAPTER 2

System Architecture

2.1 Core Technology Stack

Decision: Python is the core application language for all business logic, including the vault, encryption, request engine, queue, company database, and plugin host.
Rationale: Python allows the entire non-GUI application — the parts most relevant to security review and community contribution — to remain a single, accessible language, lowering the barrier for community code review and contribution (Sections 14 and 28).

2.2 GUI Framework

Decision: PySide6 (Qt for Python).
Rationale: PySide6 keeps the entire application in one language and process model, with no bundled browser engine to secure, update, or audit, a meaningful consideration for a security-focused tool. It is LGPL-licensed, compatible with the project's planned source-available, non-commercial licensing direction (Section 30), mature across all three initial target platforms (Section 4), and gives a native look and feel on each. This was chosen over a web UI in a Tauri/Electron shell (which would introduce a second language/toolchain and, in Electron's case, a bundled Chromium engine) and over Python-native webview frameworks such as Flet or NiceGUI (younger ecosystems, less proven at this application's scale).

2.3 Internal Component Communication

Decision: The core application (GUI, vault, queue, request engine, company database) runs in-process as a single Python application. Plugins are the sole exception and run isolated in a sandboxed subprocess.
Rationale: Full inter-process communication between every core component would add complexity with no real security payoff. The GUI, vault, and queue are all trusted, first-party code reviewed under the same process (Section 28). The one place isolation genuinely matters is plugins, which are third-party, community-contributed code operating under a default-no-access permission model (Section 21). Isolating only that boundary satisfies Phase 5's sandboxing requirement without paying an IPC-complexity cost everywhere else.

2.4 Plugin Sandboxing Mechanism

Decision: Plugins run as an isolated OS-level subprocess and interact with the core application only through a narrow, explicit capability API (e.g. discrete calls such as requesting network access or reading a specific identity profile). Stronger OS-level hardening (seccomp/AppArmor/Windows job objects) and/or a future move to WASM-based plugins are documented as a post-v1 hardening path, not a v1 requirement.
Rationale: A subprocess plus a deliberately small, permissioned API delivers the two things the specification's plugin model actually requires (Section 21): a hard process boundary so a misbehaving plugin cannot take down the vault or GUI, and an enforceable choke point where every capability is something the user explicitly granted. It keeps plugins written in plain Python, which matters for community contribution (Section 14). Full OS-level sandboxing profiles are more robust against a deliberately malicious plugin author, but require three separate, ongoing security profiles across Windows/Linux/macOS (Section 4), a maintenance cost not justified before the plugin ecosystem exists. WASM-based plugins would offer the strongest, most OS-agnostic isolation, but would force plugin authors into WASM-compilable languages, cutting against the ease of community contribution this phase otherwise optimizes for.

2.5 Concurrency Model

Decision: asyncio as the backbone for the queue and all background/network tasks (email sending, IMAP polling, scheduled checks, database synchronization), with a worker thread pool for blocking or CPU-bound operations (key derivation, document redaction, plugin subprocess calls).
Rationale: The queue's workload (Section 18) is overwhelmingly I/O-bound, which asyncio handles with far less overhead than a thread-per-task model. Relevant given bulk actions across many companies at once (Section 12) may mean dozens of concurrent waiting operations on a user's own desktop hardware. asyncio also composes cleanly with an embedded SQLite-backed queue via aiosqlite. The thread pool exists because a small number of operations are deliberately CPU-heavy (Argon2id key derivation) or otherwise blocking, and must never be allowed to freeze the GUI. Plain threads for everything were ruled out as they scale poorly under many simultaneous waiting operations; a full external task-queue system (e.g. Celery/RQ) was ruled out as it requires a broker service, directly against the dependency-minimization reasoning that already led to choosing an embedded queue over PostgreSQL (Section 18).

2.6 Cryptography Library

Decision: Python's cryptography package (pyca/cryptography) for AEAD encryption and digital signatures, and argon2-cffi for key derivation. No custom or hand-rolled cryptographic constructions anywhere in the application.
Rationale: Mapped onto the key-slot model in Section 6: Argon2id (argon2-cffi) derives the key that wraps the Data Encryption Key from a password or recovery key;

the DEK itself is generated via Python's secrets module (CSPRNG); vault contents and wrapped DEKs are encrypted with an AEAD cipher (XChaCha20-Poly1305, preferred for its larger nonce space given the DEK is re-wrapped repeatedly during password changes and recovery-key regeneration. AES-256-GCM remains an acceptable alternative); and Ed25519 signs software updates, company-database sync packages, and backups (Sections 21, 25, 26). The cryptography package was chosen over PyNaCl primarily for ecosystem weight. It is the de facto Python standard, more likely already a transitive dependency via other libraries (e.g. TLS), avoiding two overlapping crypto dependencies. Argon2id was chosen over bcrypt/scrypt as the current OWASP-recommended default with the best resistance to GPU-parallel attacks.

2.7 Packaging and Distribution

Decision: Nuitka compiles the application to native machine code for release builds, with PyInstaller retained as a documented per-platform fallback. Platform-specific distribution:

Windows: unsigned .exe/.msi installer built with Inno Setup

macOS: unsigned .dmg

Linux: Flatpak, distributed via Flathub

Rationale: Nuitka's native compilation produces a smaller, harder-to-tamper-with artifact than an interpreter-plus-scripts bundle, and is easier to hash-verify build-to-build, supporting the reproducible-builds goal in Section 26. PyInstaller remains the fallback for any dependency that resists Nuitka compilation on a given platform. On distribution: no code-signing certificate (Windows Authenticode) or Apple Developer notarization is used at this stage, as both carry ongoing monetary cost; this means Windows SmartScreen and macOS Gatekeeper will show first-run warnings, which will be documented in user-facing install instructions. A free path to remove the Windows warning exists via SignPath.io's open-source signing program and should be revisited at Phase 15. Flatpak was chosen for Linux over separate .deb/.rpm packages (avoids maintaining multiple distro-specific build targets) and over AppImage (Flatpak gives a full native-feeling install, app-store listing, icon, clean uninstall, whereas AppImage requires the user to manually mark the file executable); Flatpak's sandboxing model also reinforces rather than conflicts with the project's security positioning (Section 31).

2.8 Update Delivery

Decision: An in-app auto-updater is used for the Windows and macOS builds: the application checks a release manifest, downloads the new installer, and verifies its Ed25519 signature (reusing the signing infrastructure from Section 2.6) before applying it, consistent with the user-approval requirement in Section 26. The Linux/Flatpak build instead relies on Flatpak's native update mechanism via Flathub.
Rationale: Running a custom in-app updater alongside Flatpak's own update system would duplicate effort and fight the Flatpak sandbox, which is designed to manage its own application updates. Using each platform's most natural mechanism

keeps the update experience simple for users on every platform while preserving the signed, user-approved update flow the specification requires.

CHAPTER 3

Database and Data Model Design

3.1 Vault Storage Engine

Decision: The encrypted vault is implemented using SQLCipher (SQLite with transparent full-database encryption).
Rationale: SQLCipher provides transparent, audited full-database encryption at the SQLite layer, avoiding the need to hand-roll per-field encryption logic while still giving relational structure, indexing, and query performance for vault contents (identities, request history, rules, plugin metadata; see Section 6). This was chosen over a single encrypted blob decrypted fully into memory on unlock, which would be simpler but scales poorly as vault contents grow and forces the entire vault to be re-serialized and re-encrypted on every save rather than allowing targeted writes.

3.2 Task Queue Storage

Decision: The background task queue (Section 18) is stored in its own separate, unencrypted local SQLite file, distinct from the vault.
Rationale: Queue contents (task states, retry counts, scheduling metadata) contain no personal information and are written far more frequently than vault data. Keeping the queue outside the vault avoids unnecessary encryption overhead on high-frequency writes and keeps the vault's write pattern focused on genuinely sensitive data.

3.3 Global Company Database Format

Decision: The canonical global company database is stored as individual YAML files, one per company, each carrying an explicit version number field. This YAML source is imported into a local SQLite cache for runtime queries within the application.
Rationale: Individual per-company YAML files make Section 14's community review process practical: a single submission is a single small, human-readable diff against one file, which both human moderators and automated verification checks (Section 14.1) can evaluate in isolation. An explicit version field lets the client's sync logic detect staleness with a simple integer comparison, complementing (not replacing) the change history already provided by git. SQLite-on-import exists purely as a fast, disposable local query cache. The YAML files remain the source of truth; the imported database is rebuilt from them on each sync, not edited directly.

3.4 Global Company Database Synchronization

Decision: A public git repository holds the canonical set of company YAML files and serves as the review surface. A lightweight custom backend sits on top of it to handle submission validation, reputation-weighted voting, the 70% moderator-approval threshold, and post-approval flagging/auto-suspend (Section 14). That backend signs and publishes versioned release bundles (an archive of the currently approved YAML set plus an Ed25519 signature, per Section 26), which is what installed clients fetch and verify. End users never need git installed.
Rationale: Git provides free hosting, built-in diff/history/blame, and a natural pull-request-style review flow at no infrastructure cost, directly supporting the transparency goal in Section 31. However, git alone has no concept of reputation-weighted voting, moderator thresholds, or automatic suspension of a previously-approved-but-later-flagged record, all of which Section 14 requires; a custom backend is necessary regardless of hosting choice to implement that logic. Combining the two avoids reinventing hosting and version history while still delivering the community-governance workflow the specification describes. A fully custom sync server without git was ruled out as it would require rebuilding free infrastructure git already provides.

3.5 Local Company Database

Decision: The user's local, not-yet-globally-approved company records (Section 15) are stored in the same unencrypted local SQLite file as the imported global company cache and task queue.
Rationale: Local company records are, by the same reasoning as the global cache, not personal data as they describe companies, not the user, so no encryption benefit is lost by co-locating them. Keeping global and local company data in one file simplifies queries that need to consider both together (e.g. company search across global and local records, Section 13).

3.6 Vault Internal Schema

Decision: Identity profiles, request history, and rules are structured as normalized relational tables inside the SQLCipher vault (e.g. separate linked tables for profiles, identities, addresses, requests, and rules), rather than as flexible JSON documents.
Rationale: Normalized tables give clean referential integrity for relationships the application depends on functionally, not just descriptively, for example, which identity profile was used for a given request (Section 11's lifecycle), or which rules applied to a given bulk action (Section 12). This also makes future querying, reporting, and privacy-score calculation (Section 19) more straightforward than parsing JSON blobs at query time.

3.7 Plugin Metadata Storage

Decision: Plugin metadata, installed plugins and their granted permissions, is stored inside the encrypted vault, alongside identity and request data.

Rationale: A plugin's granted permission set reveals what personal data categories it can access, which is itself sensitive information about the user's configuration and risk exposure. Storing it inside the vault keeps that information under the same access control as the data it describes, consistent with the default-no-access plugin permission model (Section 21).

3.8 Request History Retention

Decision: Request history is retained indefinitely inside the vault by default. Users may manually prune history; no automatic retention/deletion policy is applied.
Rationale: Request history is part of the user's own record of their privacy-rights exercises and has ongoing value (tracking deadlines, referencing past correspondence, auditing which companies have been contacted, see Sections 11 and 16). Since this data lives only inside the user's own encrypted, locally-controlled vault (Section 2.1's no-central-database principle), there is no external storage-cost or exposure pressure to auto-delete it; retention control is left entirely to the user, consistent with the project's user-controlled-automation philosophy (Section 2.2).

CHAPTER 4

Repository and Development Environment

4.1 Repository Structure

Decision: A single monorepo holds the core application, GUI, and plugin SDK/examples. The company database (Section 3.4) remains its own separate repository.
Rationale: Application code, GUI code, and the plugin SDK are tightly coupled and typically change together, so a monorepo avoids cross-repo version-pinning overhead during development. The company database repository is kept separate because it has a fundamentally different contribution model, non-developer community members submitting company records (Section 14), and a different release cadence, and separating it keeps that review process from being entangled with application code review.

4.2 Branching Strategy

Decision: Git-flow: a develop branch for ongoing integration, a main branch advanced only via tagged, signed releases, and short-lived release/feature branches as needed.
Rationale: Restricting main to tagged releases gives a clean, auditable point to hook the signed-release and update-distribution pipeline defined in Phase 2 (Section 2.8). Anything on main is, by construction, something that has gone through release signing. develop gives community contributions a stable integration branch without every merge triggering a user-facing release.

4.3 Repository Hosting

Decision: GitHub.
Rationale: GitHub provides free hosting for public repositories, native issue/PR tooling that the community contribution and moderation workflows (Section 14) can build on, and integrates directly with the CI and dependency-scanning tooling chosen below, avoiding the need to run or secure separate infrastructure.

4.4 Code Formatting and Linting

Decision: Ruff, used for both formatting and linting.

Rationale: Ruff combines what would otherwise be several separate tools (Black-equivalent formatting, Flake8-equivalent linting, import sorting) into one fast, single-config tool. For a project aiming for broad community contribution (Section 14, 28), a single fast tool with one configuration file lowers the barrier to entry for outside contributors compared to maintaining Black and Flake8 (plus plugins) separately.

4.5 Static Type Checking

Decision: mypy.
Rationale: mypy is the most widely adopted type checker in the Python open-source ecosystem, with mature support for gradually typing an incrementally-built codebase and broad contributor familiarity. Pyright (which powers VS Code's Pylance) is a reasonable alternative and can be reconsidered if mypy's performance becomes a bottleneck, but mypy's ecosystem maturity makes it the safer default for a community project.

4.6 Testing Framework

Decision: pytest.
Rationale: pytest's fixture system maps well onto this project's testing needs, a reusable temporary encrypted vault, a mock plugin subprocess, a fake company database, and its parametrized-test support suits testing rules or templates against many company records at once (Sections 8, 9). It is also the de facto standard, maximizing contributor familiarity, at a trivial dependency cost over the standard library's unittest.

4.7 Continuous Integration

Decision: GitHub Actions, running formatting/lint checks (Ruff), type checks (mypy), the pytest suite, and dependency vulnerability scanning (Section 4.9) on every pull request, with matrix builds across Windows, Linux, and macOS.
Rationale: Given the repository is hosted on GitHub, GitHub Actions requires no separate infrastructure to run or secure, offers free CI minutes for public repositories, and integrates directly with PR status checks, letting the project require these checks pass before merge, and matrix-testing across all three target platforms (Section 4) in one workflow.

4.8 Pre-Commit Hooks

Decision: Pre-commit hooks run Ruff and mypy locally before every commit.
Rationale: Catching formatting, lint, and type issues before a commit is even made gives contributors faster feedback than waiting on CI, and reduces the number of "fix lint" commits cluttering PR history, keeping the git history clean and reviewable, which supports the transparency goal in Section 31.

4.9 Dependency Vulnerability Scanning

Decision: GitHub Dependabot provides passive, ongoing monitoring and automated PRs for vulnerable dependencies; pip-audit runs as an active CI check on every pull request, blocking merges that introduce a dependency with a known vulnerability.
Rationale: The project's threat model (Section 29) explicitly names compromised or malicious update pushes and supply-chain risk as threats it must protect against; a vulnerable dependency is a direct instance of that risk, especially given the application handles cryptography, an encrypted vault, and third-party plugin code. Dependabot alone only notifies; pairing it with a pip-audit CI gate ensures a known-vulnerable dependency cannot be merged silently, treating this as a required control rather than an optional one.

4.10 Issue and Pull Request Templates

Decision: Structured GitHub issue templates for bug reports, feature requests, and company-database corrections, plus a pull request template with a contribution checklist.
Rationale: A dedicated company-database-correction template does much of Section 14.1's independent-verification legwork up front, by requiring structured evidence (e.g. company name, source link, what is changing) at submission time rather than leaving moderators to extract that information from free-form reports. This matters for scaling community review as contribution volume grows. A PR checklist reinforces the formatting/type/test expectations already enforced by CI and pre-commit hooks, making expectations explicit for first-time contributors.

CHAPTER 5

Security Foundation

5.1 Application Configuration Storage

Decision: Non-personal application configuration (window state, sync settings, UI preferences) is stored in a plain, unencrypted TOML file.
Rationale: This follows the same reasoning already applied to the task queue and company database (Sections 3.2, 3.5): none of this data is personal or sensitive, so encrypting it would add overhead without a corresponding security benefit, and TOML keeps the format simple and human-readable for the user's own configuration.

5.2 Password Strength Validation

Decision: Password strength is validated using zxcvbn.
Rationale: Section 6 requires rejecting not just short passwords but common words, predictable substitutions, and simple leetspeak modifications. Simple rule-based checks (length, character-class counts) cannot catch these patterns. A password like "P@ssw0rd123!" satisfies every rule-based check while being trivially guessable. zxcvbn models realistic attacker guessing strategies (dictionaries, keyboard patterns, common substitutions) and gives a genuine strength estimate rather than a superficial rule pass.

5.3 Vault Auto-Lock

Decision: The vault automatically locks after a configurable idle timeout. Auto-lock is blocked entirely while any background task requiring vault access is active (e.g. a request mid-send), resuming its idle countdown once no such task is running.
Rationale: An idle timeout reduces the window an unlocked vault is exposed on a machine the user has stepped away from, directly supporting the encrypted-vault threat model (Section 29). Blocking auto-lock during active vault-dependent background work avoids corrupting or interrupting an in-progress operation (such as a partially-sent request, Section 11) purely because of a timer, which would be a worse outcome than a slightly longer unlock window in that specific case.

5.4 Secure Logging

Decision: Structured logging via structlog, writing to a local rotating log file. The default log stream contains no personal information, matching the format described

in Section 23. The optional personal-log stream (containing PII such as emails, company names) is implemented as a separately-gated stream that must be explicitly enabled by the user.
Rationale: Structured logging makes it straightforward to enforce, in code, exactly which fields are permitted in the default log stream, rather than relying on developers manually avoiding PII string interpolation in every log call. A separately-gated stream for the personal log (Section 23) keeps the opt-in nature of PII logging structurally enforced rather than just documented.

5.5 Plugin Permission Enforcement

Decision: The plugin permission framework is implemented as a capability-token model: a plugin subprocess only ever receives function references (RPC stubs) for the specific data and actions the user has granted. Capabilities the user has not granted are not merely denied when called. The corresponding function reference does not exist within the plugin's scope at all.
Rationale: This builds directly on the subprocess-isolation decision from Phase 2 (Section 2.4): since plugins already communicate with the core application over a narrow API boundary, that boundary is the natural enforcement point. A capability-token model is a structural guarantee. There is no code path to accidentally miss a permission check on a new function, because an ungranted capability simply has no corresponding function available to call. This was chosen over a permission-check decorator/gate model, where every sensitive call path must have the correct check applied and kept correct as the codebase grows; a single missed decorator on one new function would constitute a silent permission bypass. The tradeoff is more up-front API design work, since every plugin capability must be modeled as its own narrowly-scoped function rather than a single generic gated call. An acceptable cost for the stronger guarantee, consistent with Section 21's "default: no access" requirement.

5.6 Argon2id Parameter Calibration

Decision: Argon2id parameters (memory and iteration cost) are benchmarked and calibrated on the user's own hardware at vault creation, targeting an unlock time of approximately 0.5–1 second, with a safety floor below which parameters will not be reduced regardless of benchmark results.
Rationale: A single fixed OWASP-baseline parameter set is a compromise value tuned for average hardware: it under-uses available security margin on fast modern desktops (giving an attacker with a stolen vault file an easier time than necessary) while potentially being uncomfortably slow, or a source of pressure to lower parameters project-wide, on older or lower-end hardware. Calibrating per-installation, a brief one-time benchmark during vault creation, the same approach used by comparable tools such as KeePass and Bitwarden, gets each user closer to their own actual security/usability optimum. The safety floor exists to prevent the calibration from drifting below a safe minimum on unusually fast or misreported hardware.

5.7 Memory Protection for Key Material

Decision: The Data Encryption Key and any keys derived from the password or recovery key are protected in memory using OS-level memory-locking APIs (mlock on Linux/macOS, VirtualLock on Windows) to prevent them from being written to disk via swap or pagefile, and are explicitly zeroed when the vault locks or the application closes.
Rationale: The problem this addresses: while the vault is unlocked, the DEK and derived keys exist as plaintext in the application's memory (Section 6). Under normal OS memory management, if the system comes under memory pressure, any page of memory, including the page holding these keys, can be written out to swap space or a pagefile on disk. If that happens, plaintext key material could persist on disk well after the vault is locked or the application is closed, and would not be affected by locking the vault, since the vault's own encryption is irrelevant once the key itself has leaked onto unencrypted disk. This directly undermines the "stolen encrypted vault" and device-compromise threat categories the specification already names (Section 29): if an attacker later obtains the disk (including swap), they could obtain the key that was believed to only exist in memory. Pros of implementing OS-level memory locking and explicit zeroing:

- Prevents the DEK and derived keys from ever being written to disk via swap/-pagefile while the vault is unlocked.
- Explicit zeroing on lock or shutdown shrinks the window during which key material remains in memory at all.
- Directly closes a gap against threat categories the specification already commits to defending against (Section 29).
- Consistent with the project's positioning as a serious security application (Section 31), where this class of protection is a reasonable baseline expectation.

Cons and limitations of this approach:

- Requires platform-specific code (separate implementations for mlock and VirtualLock), adding a small, bounded maintenance surface rather than a one-line change.
- Python's memory model, including reference counting and garbage collection, makes it difficult to guarantee that no copy of a key ever exists outside the explicitly-tracked and zeroed memory region. For example, an intermediate value produced during a cryptographic operation could briefly exist in memory the application does not directly control.
- As a result, this measure substantially reduces exposure but cannot provide an absolute guarantee against key material touching disk, in the way it could in a language with fully manual memory management.

This is an active area of practical cryptographic engineering, and the specific technique used here (OS-level locking plus explicit zeroing) reflects current best-effort practice within a garbage-collected language, not a solved problem with a perfect guarantee. This decision should be treated as revisitable: it may change as better techniques become available, as the project's understanding of CPython's memory behavior deep-

ens, or as ongoing research into secure memory handling in managed-memory languages evolves.

5.8 Failed Unlock Attempt Rate Limiting

Decision: Repeated failed vault-unlock attempts trigger exponential backoff, temporarily locking out further attempts for increasing periods.
Rationale: Argon2id's computational cost already slows brute-force attempts against a stolen vault file, but that cost applies equally to an offline attacker with unlimited compute time and to a legitimate user typing a password directly into the running application. Rate-limiting attempts at the application layer adds a second, independent layer of protection specifically against an attacker attempting to guess the password through the running application itself, rather than against the vault file offline.

CHAPTER 6

Core Application Framework

6.1 GUI Navigation Pattern

Decision: A single main window with sidebar navigation between the application's main sections (Requests, Companies, Identity Profiles, Rules, Plugins, Logs, Settings, per Section 27).
Rationale: A sidebar keeps every top-level section visible and reachable in one click regardless of which section is active, which suits an application with a moderate, fixed number of top-level sections that a user will move between frequently during a session. This matches Section 27's description of "one advanced interface" better than a top tab-bar, which tends to suit a smaller or more dynamic set of views.

6.2 Settings Application Behavior

Decision: Settings changes apply live/immediately wherever possible. A restart is required only where a setting genuinely cannot be changed at runtime (e.g. certain GUI framework-level or startup-time configuration).
Rationale: Requiring a restart for every settings change would be a needless friction point for a desktop application used frequently throughout the day. Applying changes live keeps the experience closer to "as simple as possible" (Section 5), while the exception for genuinely restart-only settings is a practical necessity rather than a default policy.

6.3 Update Check Timing

Decision: The application checks for both software and company-database updates on every launch, prompting the user for approval before installing anything, exactly as described in Section 26.
Rationale: This directly implements the behavior already specified in Section 26 rather than introducing a new policy. Checking only at launch (rather than also periodically in the background) keeps the update flow predictable and tied to a moment the user is already actively engaging with the application, avoiding an update prompt interrupting an in-progress task.

6.4 Backup Framework Scope

Decision: The backup framework captures a full copy of the vault, application settings, and queue state, encrypted together as a single backup file, consistent with Section 25.

Rationale: Section 25 explicitly lists settings, vault, and history as backup contents. Capturing queue state alongside the vault ensures a restored backup returns the application to a fully consistent state, including in-flight or scheduled background operations, rather than a vault whose associated queue state has since diverged or been lost.

6.5 Backup Scheduling

Decision: Backups can be triggered manually by the user at any time, with an optional scheduled automatic backup the user may enable.
Rationale: Manual backup alone places the entire responsibility for remembering to back up on the user, which risks data loss for a user who simply forgets. An optional automatic schedule addresses this without removing user control. Automatic backup remains opt-in, consistent with the project's user-controlled-automation philosophy (Section 2.2), rather than a background behavior imposed by default.

6.6 Internal Event Notification

Decision: Background events (e.g. a request's status changing, a new email reply being detected) are propagated to the GUI using Qt's native signal/slot mechanism, bridged from the asyncio event loop via qasync, rather than a separately built custom event bus.
Rationale: PySide6 was already fixed as the GUI framework (Section 2.2), and Qt's signal/slot system is a mature, thread-safe publish/subscribe mechanism specifically designed to safely cross the exact boundary this application has: asyncio background tasks (Section 2.5) notifying the Qt GUI thread. Building a separate custom event bus would duplicate a problem Qt already solves correctly, for no real gain given the GUI framework is already fixed. Calling GUI methods directly from background code was ruled out, since Qt widgets are not safe to touch off the main thread and this would readily introduce thread-safety bugs.

CHAPTER 7

Identity and Privacy Manager

7.1 Privacy Rule Engine

Decision: The privacy rule engine (Section 8) represents user rules as structured, user-defined condition/action rules, evaluated by the engine. No free-form scripting or expression language is exposed to users.
Rationale: Structured rules keep every possible rule mechanically inspectable and safe to evaluate — the engine can enumerate exactly what a rule can check and what action it can trigger, which matters given rules govern sensitive behavior such as document uploads and information sharing (Section 8). A free-form scripting approach would be more flexible but introduces both a larger security surface (arbitrary user-authored logic running against sensitive data) and a much higher bar for a non-technical user to safely author or audit a rule, which conflicts with the strict-by-default posture Section 8 already establishes.

7.2 Privacy Score Calculation

Decision: The privacy score (Section 19) is calculated using a weighted scoring model: each data category and condition present in a given request (e.g. email, phone number, government ID requirement, manual-approval requirement) contributes a defined sensitivity weight, and the request's overall score is derived from the combination of weights actually present in that specific request.
Rationale: Section 19's own examples show the score varying based on exactly which fields a request shares, not on a generic request type — a weighted model is the only approach that can produce those example outputs ("Email only" vs. "Government ID required" plus "Manual approval required") from first principles. A fixed lookup table keyed by request type would either need a row for every possible field combination, which is impractical, or would collapse meaningfully different requests into the same score, undermining the purpose of the score.

7.3 Data Minimization Engine

Decision: The data minimization engine (Section 7) computes the minimum field set to include in a request as the intersection of the fields a company record declares as required (Section 13) and the fields the user's selected identity profile and active rules permit to be shared. Any field outside that intersection is never included in the outgoing request.
Rationale: This directly implements the data minimization principle already established in Section 3 ("the system must always send the minimum amount of informa-

tion required") as a concrete, mechanical computation rather than a general guideline. The engine has no path to including a field unless it is both actually required by the company and explicitly permitted by the user's own profile/rules. This also keeps final authority with the user, consistent with the user-controlled-automation philosophy (Section 2.2): the rules side of the intersection is entirely user-defined.

7.4 Document Metadata Removal

Decision: Metadata removal (Section 20) is powered by ExifTool, bundled directly within the application as platform-specific binaries under an internal resources path (e.g. resources/exiftool/<platform>/) and invoked via subprocess, rather than relying on a system-installed or PATH-resolved copy.
Rationale: Bundling a known-good ExifTool binary per platform guarantees consistent, predictable metadata-stripping behavior regardless of what the user has or has not installed on their system, and avoids metadata removal silently failing or behaving inconsistently due to a missing or differently-versioned system installation. Resolving the binary from an internal resources path, keyed by platform, keeps this reliable across Windows, Linux, and macOS (Section 4) without depending on the user's PATH configuration.

7.5 Redaction, Cropping, and Watermarking

Decision: PyMuPDF (fitz) powers redaction and cropping of PDF documents; Pillow powers redaction, cropping, and watermarking of image files. These are used in addition to, not in place of, the bundled ExifTool metadata-removal step (Section 7.4).
Rationale: PDF and image redaction/cropping/watermarking are meaningfully different problems from metadata stripping. They require manipulating visual content and page/pixel structure rather than reading or clearing metadata fields and ExifTool is not designed for this kind of content editing. PyMuPDF is a mature, well-supported library for PDF-specific manipulation (including true redaction that removes underlying content, not just visual overlay), and Pillow is the standard, broadly-supported choice for image manipulation in Python, keeping this step within the existing Python-first stack (Section 2.1) rather than introducing another bundled external binary.

CHAPTER 8

Company Database System

8.1 Company Search

Decision: Company search (Section 13) is implemented using SQLite FTS5 full-text search over company names, domains, and known aliases, indexed against the local SQLite cache (Section 3.3).
Rationale: FTS5 is built directly into SQLite, so it adds no new dependency beyond the database engine already chosen for the local company cache. It provides relevance ranking and prefix/typo-tolerant matching, meaningfully improving results when a user searches with a partial or slightly misspelled company name, and its indexes can be rebuilt alongside the existing YAML-to-SQLite import step. This was chosen over simple LIKE/substring queries, which offer no relevance ranking and degrade in performance as the company database grows, since substring matches cannot use a standard index.

8.2 Company Categorization

Decision: Companies are categorized using a fixed taxonomy of categories (e.g. data broker, people-search, advertiser, retailer) defined directly in the YAML schema for company records.
Rationale: A fixed taxonomy keeps categorization consistent and machine-usable across the entire global database, supporting bulk actions grouped by category (Section 12, e.g. ”remove me from all known people-search websites”) and predictable filtering/search. Free-form, community-defined tags would fragment over time into near-duplicate or inconsistent labels (e.g. ”data broker” vs ”data-broker” vs ”broker”), undermining the reliability of category-based bulk actions.

8.3 Global Company Database Sync Timing

Decision: The global company database sync runs automatically on every application launch, alongside the software/database update check described in Section 26, and is also available as a manual ”sync now” action at any time.
Rationale: Launch-time syncing keeps the local company cache reasonably current without requiring the user to think about it, consistent with the update-check timing already established in Section 6.3. A manual sync option is added on top for cases where a user wants the latest data immediately, for instance, right after submitting or hearing about a correction, without needing to restart the application.

8.4 Company Submission Interface

Decision: The in-app company submission interface produces a signed submission sent directly to the custom backend defined in Section 3.4, which validates it and opens the corresponding pull request or commit against the company database repository on the user's behalf. Users are not required to have a GitHub account, use git, or interact with the repository directly.
Rationale: This directly implements the behavior already described in Section 14, "the software converts submissions into the correct database format", as an automatic, in-app step rather than leaving the user to manually construct and submit a patch file via GitHub. Removing the need for git or GitHub literacy meaningfully lowers the barrier to contribution for the non-developer community members Section 14 is written for, which matters directly for the community-maintenance priority (Section 2.3). The tradeoff is that the backend must securely hold or be granted a credential capable of opening pull requests/commits, which is an accepted extension of the trust already placed in that backend component for validation, voting, and flagging (Section 3.4).

8.5 Local Edit Conflict Handling

Decision: If a global sync updates a company record for which the user has a pending, unsynced local edit or submission, the two versions are never silently merged. The local edit is held separately, and the user is shown both versions to resolve manually.
Rationale: Silently overwriting a user's pending local correction with an incoming global update could discard work the user has not yet submitted, and silently keeping the stale local version instead of the updated global record risks the user acting on outdated company information. Surfacing both versions for manual resolution keeps the user in control of which version is correct, consistent with the user-controlled-automation philosophy (Section 2.2), and avoids the application making a judgment call about which of two conflicting records is accurate.

CHAPTER 9

Request Generation Engine

9.1 Template Composition Model

Decision: Legal request text is generated using a block-based composition system: a library of small, reusable, prewritten legal-clause blocks (e.g. requester identity statement, legal-basis citation, verification method, deadline/response-time notice), each using Jinja2 for internal variable substitution. A rule set selects and assembles the appropriate ordered blocks for a given request based on its legal framework (Section 9), request type (Section 10), and company-specific requirements, rather than maintaining one monolithic template per framework/request-type/company combination.
Rationale: A single monolithic template per combination of legal framework, request type, and company would combinatorially explode given the specification's near-term expansion targets (GDPR, UK GDPR, CCPA/CPRA, PIPEDA, LGPD, Australia's Privacy Act, see Section 9), and would force largely duplicated legal language to be maintained and reviewed separately in many places. A block-based system lets generic blocks (e.g. requester identity, verification method) be written and reviewed once and reused across every framework and request type that needs them, while framework-specific blocks (e.g. citing the correct statutory article) are added only where genuinely needed. This also makes it easier to reason about completeness: since block selection is rule-driven rather than authored freehand per template, extending support to a new legal framework is primarily a matter of writing its framework-specific blocks and reusing already-reviewed generic ones, rather than re-deriving an entire letter from scratch.

9.2 Template Layer Storage

Decision: Base legal-framework blocks are bundled with the application and versioned with software releases. Company-specific template overrides or additions are stored within the company's YAML record (Section 3.3). User-authored customizations to templates are stored inside the encrypted vault.
Rationale: Each layer is stored alongside the thing that actually owns its lifecycle and review process. Base legal blocks change on the application's own release cadence and warrant legal-accuracy review before shipping, fitting naturally into the signed-release pipeline already established (Section 2.8, Section 26). Company-specific overrides are inherently part of that company's record and benefit from the same community review, verification, and flagging process already defined for company data generally (Section 14), keeping them in the company YAML record ensures a moderator-approved correction to a company's requirements actually reaches template behavior. User customizations are private to that user and have

no place outside their own encrypted vault, consistent with the vault's role as the sole store of personal, user-controlled data (Section 2.1).

9.3 Request Preview System

Decision: A rendered preview showing the exact final content of a request, including which specific fields were included as determined by the data minimization engine (Section 7.3), is always shown to the user and is mandatory before a request can be approved. There is no skip or trusted-template bypass.
Rationale: This directly enforces the Draft Review User Approval stages of the request lifecycle (Section 11) and the principle that the system never performs sensitive actions without user approval. Given requests are now assembled from composed blocks (Section 9.1) rather than a single fixed template, an always-mandatory preview is what lets the user verify the assembled result is actually correct and complete for their specific case, rather than trusting block selection logic blindly — particularly important the first time a given framework/request-type/company combination is used.

CHAPTER 10

Email Integration

10.1 Provider Authentication

Decision: The application authenticates to Gmail and Outlook using OAuth2, their natively supported modern authentication method. Generic IMAP/SMTP providers are authenticated using app-specific passwords as the fallback method.
Rationale: OAuth2 is the standard, more secure authentication path for Gmail and Outlook, avoids the application ever handling the user's actual account password, and is the method these providers most fully support and expect going forward. App-specific passwords remain necessary as a fallback because generic IMAP/SMTP providers (Section 16) do not uniformly support OAuth2, and requiring a single authentication method across all providers would exclude legitimate providers the specification explicitly lists as supported.

10.2 Proton Mail Integration

Decision: Proton Mail support requires Proton Mail Bridge, Proton's official local application that exposes standard IMAP/SMTP access to Proton Mail accounts.
Rationale: Proton Mail does not expose standard IMAP/SMTP access directly, by design, as part of its own security model. Proton Mail Bridge is Proton's own officially supported solution for local IMAP/SMTP access, making it the natural integration path rather than reverse-engineering or depending on an unofficial API integration, which would be fragile and outside Proton's supported surface.

10.3 Verification and Deadline Detection

Decision: Detection of verification requests and parsing of response deadlines from incoming email is implemented using rule-based keyword and pattern matching, configurable and extensible via company records (Section 13).
Rationale: Rule-based matching is deterministic and auditable. A user or moderator can inspect exactly which patterns triggered a given detection, which matters for a security-focused tool acting on the user's behalf (Section 31). Making these patterns extensible via company records lets company-specific response phrasing be captured as part of the same community-reviewed data already used for other company information (Section 14), rather than requiring an application code change for every company's particular wording. A local AI-assisted classifier remains permitted under the project's AI policy (see Section 22: optional, local-only, permission-controlled) and could be considered as a future enhancement, but is not required for this functionality to work correctly.

10.4 IMAP/SMTP Protocol Handling

Decision: The underlying IMAP and SMTP protocol work is handled directly using Python's standard library (imaplib and smtplib), rather than a higher-level third-party wrapper library.
Rationale: Using the standard library avoids adding a third-party dependency for core email connectivity, functionality the application depends on for one of its primary purposes, keeping the dependency surface smaller and easier to audit, consistent with the project's general preference for minimizing dependencies (Section 18) and its security-conscious posture (Section 28, 31).

10.5 Conversation Threading

Decision: Email conversation threading is tracked primarily using standard Message-ID, References, and In-Reply-To headers. Custom heuristics based on subject line and sender matching are used as a fallback when these headers are missing or broken.
Rationale: Standard threading headers are the most reliable and widely supported mechanism for associating replies with their original message, and using them as the primary method avoids reinventing a solved problem. The heuristic fallback exists because not every company's mail system reliably preserves these headers in practice, and without a fallback, a broken-header reply would otherwise appear as an orphaned message disconnected from its request history (Section 16), undermining the request lifecycle tracking described in Section 11.

10.6 Bounce Detection

Decision: Bounce detection is implemented by parsing standard bounce/Delivery Status Notification (DSN) messages per RFC 3464.
Rationale: RFC 3464 defines a standardized, structured format for delivery failure notifications, which allows bounce detection to be based on parsing an actual, well-defined message structure rather than a heuristic guess. A simpler heuristic based on sender address patterns (e.g. anything resembling mailer-daemon) is less reliable, since bounce sender addresses vary across mail systems and such a heuristic could both miss genuine bounces and misclassify legitimate replies.

CHAPTER 11

Automation Framework

11.1 Browser Automation Engine

Decision: Playwright powers browser automation (Section 17).
Rationale: Playwright is actively developed, supports multiple browser engines, and handles modern, JavaScript-heavy sites more reliably than older tooling, relevant given company privacy-request pages vary widely in how they are built. This gives the automation framework a solid technical foundation for both full and assisted automation (Section 17).

11.2 Automation Visibility

Decision: Browser automation runs in a visible, app-controlled browser window that the user can watch and intervene in at any point, rather than running headlessly in the background.
Rationale: A visible window lets the user directly observe what an automated action is doing on their behalf at any time, which reinforces the project's transparency principle (Section 31) beyond just being a technical necessity for assisted and manual steps (Section 17). It also gives the user a natural, immediate point of intervention if something looks wrong, consistent with the user-controlled-automation philosophy (Section 2.2).

11.3 CAPTCHA Detection

Decision: CAPTCHA detection uses heuristic recognition of known CAPTCHA provider DOM elements and scripts (e.g. reCAPTCHA, hCaptcha). When detected, automation automatically pauses and hands control to the user.
Rationale: Section 17 explicitly requires that the system must never bypass CAPTCHA. Heuristic detection of known provider markup allows the automation framework to proactively recognize a CAPTCHA and hand off to the user before any bypass attempt could even be considered, rather than relying on the user to notice a stalled or failed automation and intervene after the fact.

11.4 Company-Specific Automation Structure

Decision: Company-specific browser automation (the "specialized company automation" plugins described in Section 21) is defined declaratively as a sequence of selectors and actions, stored either in the company record or in a plugin, and interpreted

by a single shared Playwright driver rather than each company automation being its own arbitrary code driving Playwright directly.
Rationale: A declarative action sequence is far easier for community moderators to review for safety and correctness than arbitrary executable code (Section 14, 28), since its full behavior is enumerable from its declared steps rather than requiring a full code audit. It also keeps every company automation running through the same shared driver, so improvements to CAPTCHA detection, rate limiting (Section 11.6), and error handling apply uniformly across all company automations rather than needing to be reimplemented per plugin.

11.5 Automation Status Reporting

Decision: Automation status (e.g. running, paused, waiting for user, CAPTCHA-blocked, failed) is reported through the same event bus and queue status model already used for request status and background tasks generally (Sections 6.6, 11), with additional automation-specific states added to that existing model.
Rationale: Automation is, functionally, another kind of background task the queue already tracks (Section 18), and reusing the existing Qt signal/slot-based event bus (Section 6.6) means the user has a single, consistent place to see everything happening in the background, for example email sending, database sync, and automation alike, rather than a separate, parallel status system to build, maintain, and for the user to learn.

11.6 Automation Rate Limiting

Decision: The automation framework enforces per-domain rate limiting and politeness delays between automated actions, configurable by the user and defaulting to conservative values.
Rationale: Section 17 requires that automation respect a target site's Terms of Service and robots.txt directives and never resemble unauthorized scraping. Built-in, conservative-by-default rate limiting makes this a structural property of the automation framework itself rather than something each company-specific automation sequence must independently remember to implement correctly, reducing the risk of an automation inadvertently behaving in a way that could be mistaken for abusive traffic.

11.7 Manual Assistance Mode

Decision: Manual assistance (Section 17) uses the same Playwright-controlled browser window used for full and assisted automation. For manual steps, the application simply steps back and lets the user drive the browser directly, rather than opening a separate plain, non-automated browser.
Rationale: Using one browser context across all three automation approaches (full, assisted, manual) keeps the user's session, cookies, and page state continuous across a single privacy request even if it moves between automated and manual steps, for example an automated form-fill followed by a manual CAPTCHA solve followed by

an automated submission. A separate plain browser would break that continuity and could require the user to re-authenticate or re-navigate unnecessarily.

CHAPTER 12

Plugin System

12.1 Plugin Signing and Trust Model

Decision: Plugins are distributed as packages signed by their individual developer using their own Ed25519 key (consistent with Section 2.6's signing infrastructure). The project maintains and curates a registry of trusted author public keys, rather than re-signing every plugin release itself.
Rationale: A developer-signed model with a project-maintained trust registry lets the project gate initial trust (adding an author's key to the registry) without becoming a mandatory bottleneck for every subsequent plugin release from an already-trusted author. This mirrors the governance pattern already established for the community company database (Section 14): a registry the project curates, but content the community produces and updates independently. A model where the project re-signs every plugin centrally would concentrate a large, ongoing review workload in one place and work against the community-maintenance and scalability goals already reflected elsewhere in the specification (Section 2.3).

12.2 Permission Granting

Decision: Each plugin ships with a manifest file declaring every capability it requests upfront. At install time, the user reviews and individually grants or denies each requested capability before the plugin can run.
Rationale: Reviewing the full set of requested capabilities at install time gives the user a complete picture of what a plugin could do before it ever executes, rather than being asked piecemeal as the plugin runs. This is consistent with Section 21's default-no-access model and the capability-token enforcement mechanism already established in Phase 5 (Section 5.5): the manifest is effectively the human-reviewable declaration of exactly which capability tokens the plugin will be issued if approved.

12.3 Plugin Management Interface

Decision: A dedicated Plugins section in the GUI (Section 27) lists all installed plugins along with their granted permissions, version, and enable/disable/uninstall controls.
Rationale: A dedicated management view keeps a plugin's granted permissions visible and reviewable at any time after installation, not just at the moment of granting, letting a user audit or revoke access later without needing to reinstall or dig through a configuration file. This also gives the flagging/suspension mechanism (Section 12.5) a natural place to surface a plugin's suspended status to the user.

12.4 Plugin Discovery and Installation

Decision: Plugins are primarily discovered and installed through a curated, in-app plugin directory. Manual installation from a local file is available as a secondary, advanced option, subject to the same signature and trusted-key verification as directory installs.
Rationale: A curated in-app directory gives most users a safe, browsable default path that only surfaces plugins signed by a registry-trusted author (Section 12.1), keeping the common case simple and consistent with ”installation should be as simple as possible” (Section 5). Manual install-from-file remains available for advanced users or plugin developers testing their own work, but does not bypass signature verification.

12.5 Revocation of Compromised Plugins or Author Keys

Decision: A live, already-installed plugin can be flagged as suspicious by any user, mirroring the post-approval flagging model already defined for the company database (Section 14). Plugins or author keys that accumulate enough flags are automatically suspended pending moderator review, regardless of their prior trusted status.
Rationale: This directly reuses a governance pattern the specification has already justified for a structurally similar problem: Section 14 notes that a record legitimate at approval time but later compromised needs to be caught and pulled from circulation quickly, regardless of prior approval. That reasoning applies at least as strongly to a plugin, since a compromised plugin is actively executing code with granted permissions on users’ machines. Relying solely on the project to notice and revoke centrally would introduce a slower response time than a security-focused application (Section 31) should accept for actively running permissioned third-party code.

12.6 Sandbox Resource Limits

Decision: The sandboxed plugin subprocess (Section 2.4) enforces CPU, memory, and execution-time limits, in addition to its existing capability-token permission restrictions (Section 5.5). A plugin that exceeds these limits or hangs past its timeout is automatically terminated.
Rationale: Capability tokens restrict what a plugin can access, but do not by themselves prevent a plugin from misbehaving in ways that don’t require any granted capability at all, for example an infinite loop, a memory leak, or a hung network call can degrade or freeze the host application regardless of what permissions were granted. Enforcing resource limits at the subprocess boundary protects the core application’s stability and responsiveness from a broader class of plugin failures than permission enforcement alone addresses, whether the cause is malicious or simply a bug in an otherwise well-intentioned plugin.

CHAPTER 13

User Experience Refinement

13.1 Accessibility Target

Decision: No formal accessibility target (e.g. WCAG-equivalent compliance) is set for v1.0. Accessibility is explicitly identified as an item to revisit in a future release.
Rationale: Given the project's phase-based, security-first development priorities (Section 32's Guiding Principle: correctness, security, and privacy before convenience or speed), setting a formal accessibility target for v1.0 would add scope to an already extensive roadmap. Deferring rather than omitting keeps accessibility as an acknowledged, planned improvement rather than an unaddressed gap, consistent with the project's transparency principle (Section 31).

13.2 First-Launch Onboarding

Decision: For v1.0, first-launch guidance is provided through documentation and the README only. A guided in-app onboarding flow (vault creation walkthrough, first identity profile, key privacy settings) is planned as part of the finished product but is explicitly deferred to a later release.
Rationale: This mirrors the accessibility decision above: onboarding is treated as a genuine part of the intended final product rather than an afterthought, but is deferred so that v1.0's scope stays focused on the security and functional foundation established in earlier phases. Documentation-based guidance is sufficient to get a user started safely in the interim, particularly given the vault-creation and password requirements (Section 6) are already enforced programmatically regardless of whether guided onboarding exists.

13.3 Documentation Maintenance and Publishing

Decision: User-facing documentation is maintained as versioned Markdown files within the code repository (Section 4.1) and published as a static site via GitHub Pages, rather than as a separately maintained wiki.
Rationale: Keeping documentation in-repo means a pull request that changes application behavior can update the relevant documentation in the same commit, keeping docs from silently drifting out of sync with the software's actual current behavior. A real risk with a wiki, which lives outside the versioned repo history and outside the review process already established in Phase 4 (Section 4.10). Publishing via GitHub Pages requires no additional hosting infrastructure, consistent with the reasoning already applied to repository hosting (Section 4.3) and CI (Section 4.7), and versioning documentation alongside releases ensures a user reading

a given version's docs is reading docs that describe that version.

13.4 Search Scope

Decision: Search remains independently scoped per section (Companies, Requests, Identity Profiles, etc.), rather than being unified into a single cross-section search entry point.
Rationale: Each section's search serves a distinct purpose against distinct underlying data (e.g. company search against the company database, Section 8.1, versus searching one's own request history), and per-section search keeps each implementation focused and simple rather than needing a shared abstraction that ranks meaningfully different content types against each other.

13.5 Error Handling and Bug Reporting

Decision: A dedicated in-app error reporting flow shows errors to the user with a clear message and an optional "copy diagnostic info" action, sourced from the default, non-personal log stream (Section 5.4), which the user can attach when filing a GitHub issue using the bug report template (Section 4.10).
Rationale: Surfacing diagnostic information sourced only from the non-personal log stream lets a user provide genuinely useful debugging context without needing to manually locate log files or risk pasting personal information (from the separately-gated personal log, Section 5.4) into a public issue tracker by mistake. Connecting this directly to the existing bug report template (Section 4.10) keeps error reporting consistent with the structured contribution workflow already established rather than introducing a separate, ad hoc reporting path.

CHAPTER 14

Testing and Service Review

14.1 Test Coverage Policy

Decision: Test coverage is tracked and reported in CI (via pytest-cov) but is not enforced as a hard, merge-blocking threshold.
Rationale: Coverage percentage is a weak proxy for test quality — it is straightforward to reach a high number with shallow tests that execute code without meaningfully asserting on security-relevant behavior (for example, a test that unlocks the vault but never verifies a wrong password is correctly rejected). A hard threshold can perversely incentivize low-value tests written just to satisfy the number. Given this project's highest-stakes code is exactly where coverage-as-a-number is least meaningful (vault, cryptography, permission enforcement — Sections 5.5–5.7), tracked-but-unenforced coverage keeps the signal visible to reviewers while relying on actual reviewer judgment for security-sensitive PRs, rather than a mechanical pass/fail rule.

14.2 End-to-End and GUI Testing

Decision: Automated end-to-end and GUI testing is implemented using pytest-qt, testing PySide6 widgets and user flows directly within the existing pytest-based test suite (Section 4.6).
Rationale: pytest-qt integrates directly with the testing framework already chosen (Section 4.6), avoiding a second, separate testing tool or framework for GUI flows specifically. Automated GUI testing catches regressions in critical user-facing flows (e.g. vault unlock, request approval) that unit tests on underlying logic alone would not exercise.

14.3 Vulnerability Disclosure Process

Decision: The project publishes a formal vulnerability disclosure policy (SECURITY.md) describing how to report a vulnerability, a committed response-time expectation, and a credit policy for reporters. No paid bug bounty program is offered for v1.0.
Rationale: A clear, published disclosure policy is a low-cost, high-value trust signal consistent with the project's positioning as a serious security application (Section 31) and its stated encouragement of vulnerability reports and security testing (Section 28). A paid bug bounty is a meaningful ongoing financial commitment that is not necessary to establish a functioning, credible disclosure process, and can be reconsidered later if the project's resources and user base grow.

14.4 Automated Static Security Analysis

Decision: No automated static security analysis tool (e.g. Bandit, Semgrep) is run in CI for v1.0. Security review relies on manual code review, consistent with the review practices already established in Phase 4.
Rationale: This keeps the v1.0 CI pipeline (Section 4.7) focused on the tooling already established (Ruff, mypy, pytest, dependency scanning) without adding an additional automated gate before the project has evaluated whether the signal-to-noise ratio of static security analysis tooling is worthwhile for its specific codebase. This is a reasonable v1.0 scope decision rather than a permanent one, and should be revisited as the codebase and contributor base grow.

14.5 Fuzz Testing

Decision: For v1.0, property-based/fuzz testing (via Hypothesis) targets the global company-record YAML importer (Section 3.3) specifically. Fuzz testing of email/DSN response parsing (Section 10.6) and the request template engine (Section 9.1) is identified as a planned expansion for a later version, not included in v1.0 scope.
Rationale: The YAML company-record importer is the highest-priority fuzzing target for v1.0 because it processes community-submitted data (Section 14) that, while subject to review and verification (Section 3.4), is still less trusted than the application's own first-party code, malformed or adversarially crafted YAML is a plausible real input this parser must handle safely. Email/DSN parsing and the template engine also process externally influenced input and are reasonable future fuzzing targets, but are deferred to keep v1.0's testing scope focused on the highest-risk parser first; this deferral is stated explicitly here rather than left implicit, consistent with the project's transparency principle (Section 31).

14.6 Performance Testing

Decision: No dedicated performance testing (automated or manual benchmarking) is conducted for v1.0.
Rationale: Consistent with the project's overall priority order (Section 2.3, where speed ranks last), formal performance testing is not treated as a v1.0 requirement. Functional correctness and security testing (Sections 14.1–14.5) take priority within this phase's scope.

14.7 Pre-Release Community Review Window

Decision: There is no formal, separately time-boxed community security review period held specifically before the v1.0 tag. Community review is continuous and ongoing throughout development, via the standard pull request review process and issue reporting already established (Sections 4.2, 4.10).
Rationale: Since every change already goes through pull request review before merging to develop, and community members can report issues at any time via the established issue templates (Section 4.10), a review process is already continuously

active rather than concentrated into a single pre-release window. Introducing a separate formal window would add process overhead without a clear corresponding gain, given review is not being deferred to that window in the first place.

14.8 Release Candidate Promotion Criteria

Decision: A release candidate is promoted to the public v1.0 release only once all known issues, of any severity, have been resolved.
Rationale: This reflects the project's stated guiding principle (Section 32) that correctness, security, and privacy take precedence over rapid feature delivery or release speed at every stage of development. Requiring all known issues, not only critical or high-severity ones, to be resolved before the first public release sets a deliberately high bar for the version most new users will judge the project's trustworthiness by.

CHAPTER 15

Public Release

15.1 Versioning Scheme

Decision: The project uses Semantic Versioning (MAJOR.MINOR.PATCH), with the public release starting at 1.0.0.
Rationale: Semantic Versioning gives users and contributors an immediately meaningful signal about the nature of a given update, a MAJOR bump signals breaking changes, MINOR signals new functionality, PATCH signals fixes, which matters for a security-sensitive desktop application where users need to judge how significant an update is before approving it (Section 26). Calendar versioning would convey release timing but not the nature or risk of a given change, which is the more relevant signal for this project's update-approval flow.

15.2 Release Binary Distribution

Decision: Signed Windows and macOS installers/bundles are published as attachments on tagged GitHub Releases, serving as the canonical download source, linked from the documentation site (Section 13.3). The Linux build continues to be distributed via Flathub, as already established in Section 2.7.
Rationale: Publishing through GitHub Releases keeps release distribution on the same platform already used for the repository, CI, and issue tracking (Sections 4.3, 4.7), avoiding the need to build and secure a separate release-hosting website. Tying releases directly to git tags also keeps the signed-release pipeline (Section 2.8) anchored to a single, auditable point of truth, consistent with the git-flow branching model already established (Section 4.2), where main only advances via tagged releases.

15.3 Post-Release Roadmap Publication

Decision: The public post-v1.0 roadmap is maintained in two complementary forms: a living ROADMAP.md file in the repository for high-level direction and priorities, and GitHub Issues/Milestones for granular, trackable feature and fix planning.
Rationale: A ROADMAP.md gives prospective users and contributors a quick, readable sense of where the project is heading without needing to parse dozens of individual issues, while GitHub Milestones give contributors a concrete, actionable view of what is planned for a specific upcoming release. Using both keeps high-level direction and granular tracking each in the format best suited to it, rather than forcing one document to serve both purposes.

15.4 Timing of External Community Contributions

This item remains an open decision rather than a settled one. Three plausible approaches, evaluated against the project's philosophy, are recorded here for future decision:

- Open immediately at v1.0 launch; maximizes early community involvement and aligns most directly with the community-maintenance priority (Section 2.3), but means the project's review capacity (Section 4.10) is tested by external contributions before the codebase and processes have had time to stabilize post-launch.
- Closer maintainer scrutiny before v1.0, formalizing into the standard PR process afterward. Gives the maintainer(s) more control while the security foundation and core architecture are still settling, reducing the risk of reviewing external changes against a moving target, but delays the community-maintenance benefits the project is explicitly built around.
- A hybrid: accept issue reports and discussion pre-v1.0, but hold external code contributions (PRs) until after v1.0; balances early community engagement (bug reports, company-database corrections, which are lower-risk than code changes) against keeping code review load manageable while the codebase is least stable. This should be revisited and finalized as v1.0 approaches and actual maintainer review capacity becomes clearer.

15.5 License Publication

Decision: The exact PolyForm Noncommercial 1.0.0 license text is published as a LICENSE file at v1.0 launch. Section 30's licensing rationale, explaining why a source-available, non-commercial license was chosen over an OSI-approved open-source license, is republished as a standalone companion document (e.g. LICENSING.md), linked from the README, rather than left only inside this specification.
Rationale: Section 30 already commits to the rationale being "made publicly visible so the distinction is never misrepresented to users or contributors." Publishing the rationale as its own README-linked document at launch, rather than leaving it findable only within this internal specification, is what actually fulfills that commitment for a user or contributor encountering the project for the first time, they are unlikely to read this technical specification, but are likely to check a linked LICENSING.md if the license terms raise a question.